ORION
有限状态系统的可验证发现计划

Černý 同步问题、最短证书与信息边界

研究实操手册 · 配套代码与实验基线

从普通电脑上的第一个实例出发，建立“搜索—证书—独立检查—形式化证明”的完整研究流程，并进一步研究：一份数学证书究竟需要保留多少状态信息，才能证明一个解真正最优?

本期重点是同步自动机。研究切入口为特征压缩造成的认证能力损失：先复现精确最短长度，再用可证明的障碍与可计算的信息层级寻找更一般的规律。

起步要求	首个可见结果	本期研究核心
Python 3.10+ 不要求 GPU	C_2—C_8 最短词与证书 无需第三方 Python 包	两锚点闭式与信息损失 可继续进行 Lean 形式化

配套目录：ORION_SyncLab | 版本：v1.0 | 编写日期：2026 年 9 月 27 日

文献用于确定背景；实例数值来自配套程序实跑；形式化源码的实际验收状态在本手册中单独记录。

01 / MISSION

任务总览：你现在接手什么

本项目的长期目标，是建立可复用的有限状态系统证书方法：搜索器可以复杂，最终数学结论应当由一组明确、有限、可重复检查的条件支撑。第一期从 Cěrný 同步问题进入。该问题研究任意初始状态能否被同一个输入串带到同一状态，以及最短需要多长；经典猜想要求所有 n 状态同步自动机的阈值不超过 $(n-1)^2$。[2,3]

当前最具体的研究问题为：把一个状态集合压缩成“有多少状态、包含哪些特定位置”等信息后，还能证明多强的最短长度下界？项目已经给出精确计算基线、两项可继续形式化的纸面推导，以及一个可扩展的锚点信息层级。执行者从已有基线向前推进，不需要从空白文件开始。

组成	本包已经提供	执行者的下一项增量
计算	精确搜索、独立证书检查、局部变体扫描	替换实例，扩展实验与反例测试
数学	势函数定理、抽象最优性、两锚点及旋转障碍的纸面推导	逐步审阅并形式化；研究 r 个锚点
Lean	核心证明及 C_3/C_4 源码草案	在真实编译器中验收并完成一般语义连接
研究记录	任务提示词、日志、查重与提交模板	填入真实版本、贡献和审阅结果

验收状态 Python 部分已实跑；Lean 源码尚未在准备环境编译，不能据此宣称内核认证。本包的研究命题也未完成独立的新颖性审查。

阅读顺序：先完成第 2—3 节的首次运行，再阅读第 5—8 节的数学主线。第 10—15 节给出正式执行流程；第 18 节用于排错。完整纸面推导另存于 docs/THEORY.md。

首次运行：把第一项结果做出来

操作一：解压并找到正确位置

将整个压缩包解压到一个普通文件夹。进入 ORION_SyncLab，确认同一层能看到 run.py、START_HERE.md、src 和 tests。Windows 可在资源管理器地址栏输入 powershell 后回车；Linux 或 macOS 在该文件夹打开终端。命令必须在含有 run.py 的这一层执行。

操作二：确认 Python，然后运行三条命令

先输入 python --version，确认版本至少为 3.10。Windows 也可以用 py；Linux/macOS 常用 python3。下文统一写 python，请按本机可用命令整体替换。尚未安装 Python 时，使用官方安装说明；安装后重新打开终端。[4]

```
python --version
python run.py demo
python run.py compare --n 8
python -m unittest discover -s tests -v
```

demo 应依次显示 C2 到 C8，长度为 1、4、9、16、25、36、49，并带有 VERIFIED。compare 应显示 marked 的下界为 19、identity 的下界为 49。测试末尾应显示 Ran 14 tests 和 OK。成功后，打开新生成的 results 文件夹；每个 JSON 都是可再次检查的证书。

操作三：保留第一份实验记录

复制 templates/RESEARCH_LOG.md 作为当天日志，记录 Python 版本、上述三条命令、实际输出和运行日期。初次阶段无需 API key、CUDA、PyTorch、数据库或第三方 Python 包。Windows 也可双击 start_windows.bat；Linux/macOS 可执行 sh start_unix.sh。

显示含义 VERIFIED 仅指独立 Python 检查器接受证书。正式形式化状态以实际 Lean 编译、语义对应及评审为准。

03 / READ THE RESULT

看懂第一项结果：四个字符为何最短

先使用三个状态的实例建立直觉。a 使状态沿 0→1→2→0 循环；b 保持 0 与 1 不动，把 2 送到 0。你不知道系统最初在哪个状态，所以同时跟踪所有可能状态。一个输入字符会改变这个“可能状态集合”。

```
python run.py witness results/demo/C3.json
python run.py verify results/demo/C3.json
```

时刻	输入	可能状态集合
开始	—	{0,1,2}
1	b	{0,1}
2	a	{1,2}
3	a	{0,2}
4	b	{0}

第一条命令显示上表对应的轨迹。无论最初位于哪个状态，baab 都把系统带到 0，所以最短长度至多为 4。第二条命令检查另一组信息：所有非空集合上的势函数，以及每条转移满足的局部不等式。它同时给出最短长度至少为 4，因而上下界相等。

打开 JSON 时重点看 automaton、word、potential、lower_bound、upper_bound。word 使用字母编号：0 代表 a，1 代表 b；potential 的索引是集合的二进制掩码。例如集合{0,2}对应 1+4=5，集合{0,1,2}对应 7。索引 0 表示空集合，仅作为数组占位。

你应能复述 “我已经找到一个长度 4 的同步词，并用覆盖所有转移的下界证书排除了长度 0、1、2、3。”这句话准确表达了当前实例结论。

文件结构与研究流程

```
ORION_SyncLab/
  run.py                      # 所有常用命令入口
  src/engine.py               # 搜索与特征抽象
  src/check_certificate.py    # 独立检查器
  src/lean_export.py          # Lean 草案生成
  tests/test_lab.py           # 回归与破坏测试
  examples/                   # 可修改的输入实例
  lean/                       # 证明源码草案
  prompts/                    # 分工任务提示词
  templates/                  # 日志、查重、提交模板
  reference_results/          # 随包交付的实际运行基线
  docs/                       # 完整手册与数学笔记
```

研究流程只有一条主线：输入自动机，搜索候选同步词和势函数，把候选写成 JSON，交给独立检查器，再将需要正式提交的结论连接到形式证明。任何模型生成的猜测，都必须走同一条流程。

搜索程序使用位运算构造转移；独立检查器用集合与算术重新计算集合像，且不调用搜索器。这样的分工降低两端共享同一实现错误的风险，但仍需测试和形式化。检查器会验证输入范围、每个单点势函数、每条局部不等式、同步词终点，以及声称 exact 时上下界是否相等。

reference_results 是已有基线，应保留不变。自己的新运行写入 results。修改代码前，先复制原始文件夹或建立 Git 提交；后续写成果说明时，可以精确指出新增了哪个定理、实验或接口。

```
python src/check_certificate.py results/demo/C3.json
python run.py solve --input examples/nonsync3.json
```

第二条命令展示不可同步实例。此时检查器验证一组包含初始集合、对每个字母封闭且不含单点的集合状态，形成不可同步的有限证据。

05 / THEOREM I

核心数学：局部检查推出全局最短性

为每个非空状态集合 S 指定一个非负整数 d(S)。你可以把它理解为：从这里开始，至少还需要多少步才能变成单点。只要下面两个条件成立，就能对任意长度、任意内容的输入串给出下界。

$$d(\{q\}) = 0 \quad \text{且} \quad d(S) \le 1 + d(S \cdot a)$$

第二个条件要求对所有非空集合与所有字母成立。它允许某一步使 d 增加，但禁止一步让 d 下降超过 1。如果起点的 d 值为 49，而终点必须为 0，那么到达终点至少需要 49 步。

完整证明

设同步词为 $w=a_1 \ldots a_L$，定义 $S_0=Q$、$S_i=S_{(i-1)} \cdot a_i$。逐次应用局部不等式得到 $d(S_0) \leqslant 1+d(S_1) \leqslant 2+d(S_2) \leqslant \ldots \leqslant L+d(S_L)$。终点为单点，因此 $d(S_L)=0$，得到 $d(Q) \leqslant L$。如果另有一个长度恰为 d(Q) 的同步词，则其上界与证书下界相同，最短性成立。该证明对所有有限输入串有效。

为何计算可以很小

有 n 个状态时，非空子集数为 2^n-1；二字母实例需要检查 $2(2^n-1)$个局部不等式。n=8 时仅有 510 项。寻找最短词可以很费力，但验证某个候选下界只需要这组确定条件。本包使用完整反向广度优先搜索产生精确势函数；这是一条实验基线。

范围 本实现显式枚举幂集，时间与内存随 n 指数增长。首轮使用 n≤10；代码输入上限为 16，它不代表所有规模都有理想运行时间。

下一项任务：阅读 lean/CertificateCore.lean 中的 potential_along_word 与 lower_bound，逐项把纸面符号对应到代码参数。核心推导只使用有限词的归纳与自然数加法。

06 / THEOREM II

研究切入口：压缩信息会损失多少证明能力

完整证书为每个子集保存一个数，但许多集合可能具有相似特征。设 α(S) 表示你选择保留的信息，例如集合大小，或集合大小加上是否包含 0 和 n−1。把特征相同的集合合并成一个抽象类，就得到一个更小的表示空间。

抽象边的定义必须精确：若存在某个真实集合 S 和字母 a，满足 α(S)=x、α(S·a)=y，就建立 x→y。目标类为单点集的特征像。令 D(x) 为抽象图上到目标类的最短距离，则 d(S)=D(α(S)) 满足上一节的局部条件。证明只有一句关键理由：先沿一条抽象边走一步，再走后继到目标的最短路径，便得到 D(x)⩽1+D(y)。

这个抽象下界已经是该特征空间的最强值

假设另一势函数 h 也只依赖 α，且满足全部局部条件。由于每条抽象边都有真实代表，h 必须在每条抽象边上满足下降至多 1。沿抽象最短路径累加就得到 h(x)⩽D(x)。因此，继续让模型在完全相同的特征上寻找更大势函数，无法超过 D。

一个重要现象：抽象图允许出现伪路径

某条抽象边可能由集合 S_1 见证，下一条边却由另一个同特征集合 S_2 见证。逐步合并后，抽象路径未必对应某条真实轨迹。它可能比任何真实同步路径都短。由于所有真实路径仍会映射为抽象路径，得到的下界依然有效；它只是失去了部分区分能力。

本项目因此有一个明确研究对象：认证下界与保留信息量之间的关系。你将先测量差距，再解释差距，最后将规律写成参数化定理。完整定义与证明见 docs/THEORY.md。

07 / THEOREM III

可直接推进的命题：两锚点闭式

固定 n≥3。对于 Černý 实例，保留 α(S)=(k,u,v)，其中 k=|S|，u 表示是否含 0，v 表示是否含 n−1。相应抽象最短距离在全体集合上的精确值为 3n−5。下面给出纸面证明；执行任务是逐项复核并完成形式化，新颖性另行审查。

特征条件	势函数 p(k,u,v)
k=1	0
k≥2，v=0	3k−3
k≥2，v=1，u=0	3k−4
k≥2，v=1，u=1	3k−5

字母 a 保持 k，并把(u,v)变为(v,t)，其中 t 表示原集合是否含 n−2。逐项代入上表，势函数下降最多 1。字母 b 在 v=0 时不改变特征；在(u,v)=(0,1)时变成(1,0)，保持 k；在(u,v)=(1,1)时变成(k−1,1,0)。最后一类转移在 k≥3 时使势函数下降 1，在 k=2 时从 1 到 0。单点情形始终为 0。因此该势函数提供有效下界 3n−5。

再证明它可达到。初始类(n,1,1)经 b 到(n−1,1,0)。对于 2≤k≤n−1，抽象图存在(k,1,0)经 a 到(k,0,1)，再经 a 到(k,1,1)，最后经 b 到(k−1,1,0)。前两条边分别可用“含 0 与 n−2、不含 n−1”和“含 n−1 与 n−2、不含 0”的 k 元集合见证。重复直到 k=1，路径长度为 1+3(n−2)=3n−5。

边界核对 必须覆盖 n=3、k=1、k=2，并只对可实现的特征类推理。3n−5 是特定抽象系统的阈值；真实 C_n 的经典阈值仍为$(n-1)^2$。[2]

源码测试已在 n=3 至 10 逐一核对每个非空子集的分段公式，并在 n=3 至 12 记录了起点数值。有限测试用于找错，普遍结论依靠上述推导。

08 / THEOREM IV

更一般的障碍：旋转不变性为何只剩线性下界

进一步限制势函数不区分整体旋转：对每个集合 S，都有 d(S·a)=d(S)。这涵盖只看集合基数、最大循环间隙、循环间隙多重集等特征的情况。即使继续使用复杂搜索，只要势函数仍具有这种不变性，起点可认证的下界就不会超过 n−1。

$$d(S\cdot a) = d(S) \quad \Rightarrow \quad d(Q) \le n-1$$

完整证明

考虑经典同步词 b(a^(n−1)b)^(n−2)。第一次 b 将全体状态变为{0,⋯,n−2}。若当前集合为{0,⋯,m−1}且 m⩾2，a^(n−1)把它变为{n−1,0,...,m−2}，随后 b 把它变为{0,⋯,m−2}。因此每一轮减少一个状态，最终成为单点。整个词总共使用 n−1 个 b。

由于 a 不改变 d，每个 b 又使 d 下降至多 1，沿这条同步轨迹累加便得到 d(Q)⩽n−1。另一方面，d(S)=|S|−1 满足全部条件：a 保持基数，b 每次最多减少一个状态。因此 n−1 确实是这个证书类的最强下界。

你从这个结论获得什么

它解释了一个容易误判的实验现象：循环间隙看起来比集合大小包含更多几何信息，但只要失去绝对位置相位，这些信息也可能无法改善起点认证下界。后续研究应尝试保留少量位置标记、添加历史信息，或换用更强的证书体系。每一种改变都需要重新写清假设。

可交付目标 把该障碍形式化成条件化定理，并展示三种特征满足旋转不变性。这样的结果明确说明一类方法的能力边界；它不等价于真实同步问题的普遍下界改进。

实际实验：第一张研究结果表已经存在

```
python run.py compare --n 8
python run.py landmarks --n 8
```

本包准备时在 C_8 上运行得到下表。真实最短同步词长度为 49。所有下界均经独立 Python 证书检查；相同的上界词同时保存在相应 JSON 中。你应先复现这张表，再修改特征，避免在不同实现间比较不一致的数据。

特征	抽象类数	认证下界	下界/真实最短
集合基数 cardinality	8	7	7/49
基数+两锚点 marked	27	19	19/49
基数+最大间隙 maxgap	20	7	7/49
基数+间隙多重集	22	7	7/49
完整掩码 identity	255	49	49/49

这张表有两个直接观察：增加特征类数量未必增加起点下界；有针对性的位置信息在此实例中比旋转不变的间隙信息更有效。第 7—8 节给出了相应结构性解释。这里的“有效”只针对指定实例与指定势函数证书类。

打开 results/compare/comparison.csv，可查看 abstract_classes、lower_bound、optimal_length、construction_seconds 和 expanded_certificate_bytes。类数衡量抽象信息规模；当前程序为验证方便仍把势函数展开到全部掩码，所以类数减少不会自动等于 JSON 体积按同一比例减少。

测量口径 当前构造抽象图仍枚举所有真实子集，不能把这张表写成渐近算法加速结果。后续若直接符号化生成抽象图，才有新的构造复杂度问题可研究。

下一项主攻任务：r 个锚点的信息层级

定义一组逐渐增加的位置标记：r=0 时不记录位置；r≥1 时依次记录 0、n−1、n−2、…、n−r+1，共 r 个位置，再加集合总基数。设 B_r(n)为该特征抽象的最短目标距离。它给你一个边界明确、可枚举、可证明的研究问题：增加多少位信息，才能恢复真实最短性的认证?

n=8 的锚点数 r	0	1	2	3	4	5	6	7	8
抽象类数	8	15	27	47	79	127	191	255	255
认证下界	7	7	19	20	21	23	25	49	49

你已经拥有三个支点：$B_0(n)=n-1$；$B_2(n)=3n-5$（$n\geq 3$）；$B_{(n-1)}(n)=(n-1)^2$。第三个结论来自一个简单事实：记录 n−1 个成员位及总基数，就能恢复最后一个成员位，因此没有信息合并。

执行顺序：只推进一个尚未证明的分支

先选择固定 r=3、n≥5。运行 n=5、6、7、8、9、10 的实验，保存全部结果，再尝试写出关于任意 n 的公式。不要同时切换到 r=n−2。若找到公式，先自动寻找反例，再构造可行势函数作为下界，并构造抽象路径作为上界。上界与下界相等才算完成精确刻画。

```
python run.py landmarks --n 5 --out results/landmarks5
python run.py landmarks --n 8 --out results/landmarks8
python run.py landmarks --n 10 --out results/landmarks10
```

再将同一组任务交给 prompts/05_research_extension.txt。该文件固定了锚点顺序与抽象边定义，能避免模型在推理中悄悄更换研究对象。有限表格中的趋势只是候选规律，待证明的量词范围必须明确写出。

Lean 环境与首次形式化验收

Python 阶段完成后，再安装 Lean。按 Lean 官方安装入口完成本机安装；项目用 lean-toolchain 固定为 leanprover/lean4:v4.19.0。这个固定版本用于复现目标，不代表赛事指定版本；提交环境若有不同要求，需要同步迁移并重新编译。[5,6]

进入项目根目录，先执行下列命令。若使用 elan，工具会依据当前目录的 lean-toolchain 选择版本。首次下载工具链需要网络。当前工程没有依赖 mathlib；后续连接其 DFA 定义时再增加并固定库提交。

```
lean --version
lean lean/CertificateCore.lean
lean lean/Cerny3.lean
lean lean/Cerny4.lean
```

正式通过标准为：命令退出码为 0，没有未解决错误；保留全部输出；核对文件末尾 #print axioms 的实际结果。普通 decide 用于有限可判定命题。未经约定不要用额外公理或运行时信任捷径替换证明；Lean 的战术说明可用于确认具体方法的行为。[7]

这些源码当前证明目标的精确范围

CertificateCore 表达一般转移系统的局部势函数下界。Cerny3 与 Cerny4 包含具体幂集系统、单步集合像对应检查、同步词和最短性定理草案。将任意词下的集合演化与一般 DFA 同步定义连接，属于下一项独立任务，不能在说明中省略。

实际状态 准备环境缺少 Lean 编译器，且未能取得工具链，因此本包没有 Lean 通过日志。源码已提供，但首次真实编译和错误修复由上述验收步骤完成。

遇到错误时使用 prompts/03_lean_engineer.txt，只修复首个报错并保持原定理不变。未经运行，不得把模型的“应当可编译”填写为通过。

执行单元一与二：复现、理解、确定唯一目标

单元一：建立可重复的工作台

先完成第 2 节的三条命令，将输出保存到自己的 results 目录。打开 C_3 轨迹，并手工计算一个集合的二进制掩码。再用独立检查器单独检查 C_3 证书。你的交付物为首份日志、Python 版本、结果目录和一段能解释上下界含义的文字。预计安排半天；这是时间预算，不是完成保证。

```
python run.py demo --max-n 10
python src/check_certificate.py results/demo/C3.json
python run.py witness results/demo/C3.json
```

单元二：锁定一项主结论

阅读 docs/THEORY.md，首选“两锚点抽象闭式 $B_2(n)=3n-5$”作为本期形式化目标；“旋转不变性障碍”作为配套引理。把完整陈述复制到日志，注明 $n \geq 3$、具体自动机方向、特征定义和存在型抽象边。不要在同一阶段同时承担一般 Černý 猜想、任意 r 闭式和大规模搜索。

使用 templates/NOVELTY_AUDIT.md 检索目标。先查来源定理与相关形式化，再查未合并 PR。至少保留最接近的三项材料、它们的精确结论和与你当前目标的差异。基础最短路证书属于标准思想，不能仅因实现方式不同就认定数学首次性。

单元	必须产生的文件	完成条件
一	当日日志、demo 结果、环境记录	长度表一致；14 项测试通过
二	目标陈述、查重记录、证明笔记	所有量词与定义固定；知道要复用什么

单人执行时，前半天由自己复现，后半天让 AI 逐条解释与审阅；两人协作时，一人负责数学陈述与来源，另一人负责代码和日志。每天结束前统一目录与版本。

执行单元三与四：把实验变为参数化证明

单元三：复现信息损失，再写反例测试

运行 C_8 的 compare 与 landmarks，将结果表写入自己的报告。随后改用 n=5 与 n=10，检查两锚点下界是否分别为 10 与 25。不要只看起点：tests 中的 test_marked_closed_form_every_subset 会核对每个非空集合的分段式。把 n=3 与 k=2 单列，写明 b 如何把势函数 1 变成 0。

```
python run.py compare --n 5 --out results/compare5
python run.py compare --n 10 --out results/compare10
python -m unittest discover -s tests -v
```

单元四：按最小引理推进形式化

先验收 CertificateCore。接着为两锚点命题建立五项引理：字母 a 的特征转移；字母 b 的特征转移；分段势函数的局部不等式；两类代表集合的存在性；长度 3n−5 的抽象路径。按这个顺序完成，最后才组合主定理。每写一项立刻编译，避免一次生成整篇证明后无法定位错误。

代表集合的存在性是关键：对于 $2 \leqslant k \leqslant n-1$，你需要从其余 n−3 个状态中补齐 k−2 个，分别得到含{0,n−2}或{n−1,n−2}的集合。n=3 时补齐数为 0。集合基数与元素排除条件必须在形式陈述中出现。

单元完成标准 纸面证明每一步有具体引理对应；至少核心下界定理和一个固定实例在真实 Lean 环境通过；新增的一般性引理有独立编译日志。

若一天内仍被某个库接口卡住，先在最小文件中证明同一辅助事实，再移回项目。不要为一个记号问题重新定义整套自动机，也不要用额外假设把原命题变得更容易。

执行单元五与六：检查器、语义与独立复核

单元五：让错误证书确实失败

原始证书通过检查仅完成正向测试。还要破坏一项条件，确认检查器拒绝它。本包已经覆盖势函数局部错误、非同步词、越界转移、被截断的势函数表、伪造不可同步不变量和上下界不一致。运行 tests 后查看对应 test_reject 名称，再至少补充三种不同的坏输入。

计算完整性还通过两条方向相反的搜索交叉核对：反向 BFS 生成精确距离，前向 BFS 从全体集合寻找单点。测试枚举全部 3^6=729 个有标号的三状态双字母完整自动机，并运行 128 个固定种子随机实例。这个范围是软件回归覆盖，不应写成所有规模的数学验证。

单元六：完成题意到形式定义的连接

对一般自动机证明：掩码表示的集合经过一个字母，恰等于逐个状态转移后的像；对词长度归纳，推广到任意输入串；最后证明单点终态等价于“存在一个状态，所有初始状态读入该词都到达它”。将这一语义桥梁与 CertificateCore 的下界定理组合，才得到可复用的一般 DFA 证书接口。

```
python run.py export-lean results/demo/C3.json
lean lean/Generated.lean
```

export-lean 当前只接受 n≤5 的二字母 exact 证书，以控制普通 decide 的展开规模。源码生成成功只意味着生成了文件。编译失败时，不得用 Python 输出替代形式证明状态；先按第 11 节修复并记录。

独立审阅 将 prompts/04_reviewer.txt 交给另一个审阅过程，要求从定义与坏输入出发检查，不沿用原作者的成功判断。保存具体反例尝试与回复。

最终交付：研究报告与赛事材料

项目首先按独立研究成果组织，再决定参赛归类。手册中的 M2 接收新的、忠实、有用且可复用的已知数学形式化，并按成果家族处理；开放问题结果有独立的准入与审查。所有计分成果需要固定形式环境、完整证据及数学审阅。基线与当期增量必须分开。[1]

赛事手册列出的窗口为 2026 年 9 月 27 日 12:00 EDT 开场，至 10 月 3 日 00:00 EDT 前提交完成；对应北京时间为 9 月 28 日 00:00 开场、10 月 3 日 12:00 截止。将准备阶段作为基线，六个执行单元压缩到这个窗口。窗口外继续研究仍有项目价值，但不能自动算作当期新增。[1]

你应提交的项目包

整理一份精确陈述与证明说明、一组实际通过的形式文件、一套可复现证书及命令、完整版本和日志，以及作者贡献和来源记录。打开 templates/SUBMISSION_PACKET.md 逐项填写。目标 ID、Hill 版本、Climb 及最终报告必须来自真实平台，不自行编造。具体入口与允许的验证环境以主办方发布内容为准。[1]

最后一次验收

从一个全新目录复制项目，按 README 完整重跑；核对报告中每项数值都对应输出文件；核对每项形式通过声明都有实际编译日志；核对原始定义、子集定义和定理陈述一致。然后固定最终提交，列出相对于基线的修改。

命名建议 研究报告可题为“Information Limits of Verifiable Synchronization Certificates”。标题覆盖实际研究对象；摘要只列确实完成的定理、实例范围与验证状态。

报告不是代码清单。要先说明为什么某类信息不足，再展示形式化如何把这个结论变成可重放证据。成果能否获得计分或发表，以实际新颖性、完整性与审阅结果决定。

更换实例与局部变体扫描

当基线完全通过后，可以直接修改输入。examples/cerny4.json 中每一行对应一个字母，每列对应一个状态；transitions[a][q]就是读入该字母后的状态。只允许 0 到 n−1 之间的整数。复制该文件再改动，避免覆盖原始样例。

```
python run.py solve --input examples/cerny4.json
python run.py verify results/certificate.json
python run.py compare --input examples/cerny4.json
```

局部变体扫描则从标准 C_n 开始，每次仅改变一个转移的目标，枚举所有其他目标。n=6 时共有 2×6×5=60 个有标号变体。运行下列命令，检查输出 CSV 与每个变体的证书。

```
python run.py mutate --n 6 --out results/mutations6
```

随包基线的实际结果为：60 个变体中，43 个同步、17 个不可同步；同步变体中的最大最短长度为 17。标准 C_6 的长度为 25，但原实例不在“必须改变一条转移”的扫描集合内。这个比较只描述指定局部邻域。

怎样使用这些数据继续研究

先挑一个最短长度较大的变体，记录变化发生在哪个字母、哪个状态、目标如何改变。随后改变 n，保持变化规则而非照搬固定编号，观察是否形成一族。若提出普遍公式，必须给出参数化定义和严格上下界。局部扫描没有进行同构去重，也没有枚举全部六状态自动机。

资源限制 mutate 默认只允许 n≤10。先从 6 开始，再逐步增加；不要同时增加状态数、字母数和变更条数，否则难以判断计算量与数学变化各来自哪里。

把任务交给 AI：可复制的工作契约

使用现有聊天或代码助手即可，无需先搭建多 Agent 平台。每轮只给一个可验收任务，并附准确文件和命令。prompts 目录已经提供五份任务契约，分别用于操作复现、数学推导、Lean 工程、独立审阅和研究扩展。

文件	何时使用	要求产物
01_operator.txt	第一次运行	真实日志、退出码与下一条命令
02_mathematician.txt	复核两锚点闭式	完整转移分类与边界证明
03_lean_engineer.txt	首次编译及报错修复	实际通过日志与公理依赖
04_reviewer.txt	主结果完成后	错误输入、缺口与具体修改
05_research_extension.txt	研究 r=3 或 r=n−2	精确猜测、反例测试及剩余引理

一次任务的最小输入

提供项目目录或相关文件，说明当前固定版本，附最近一次实际报错，最后只要求完成一个目标。例如：“保持定理陈述不变，只修复 CertificateCore.lean 的首个编译错误，给出实际执行命令与退出码。”数学任务则要求先写清假设，再提出引理，最后逐步证明。

不要把完整手册反复发给所有模型。操作模型只读入口和检查器；数学模型读 THEORY 及相关测试；证明模型读目标 Lean 文件；审阅模型读最终陈述、证书与日志。这能减少上下文混淆，也便于追踪哪个修改来自哪一项任务。

人需要做的检查 确认模型没有改量词、没有把抽象系统换成真实自动机、没有把有限测试写成普遍定理，并亲自运行最终命令。

故障处理：看到什么，就执行什么

现象	先检查	明确动作
找不到 python	解释器是否已安装	Windows 试 py；Linux/macOS 试 python3；按[4]安装后重开终端
找不到 run.py	当前目录	回到含 run.py 的 ORION_SyncLab 根目录
ModuleNotFoundError: src	是否只复制了单个脚本	重新解压完整项目，在根目录运行
JSON 解析失败	逗号、引号、文件编码	从 examples 复制原始文件，只改一个整数再试
n 或转移越界	编号是否从 0 开始	每行长度等于 n；目标必须在 0…n−1
VERIFICATION FAILED	证书是否被修改	先检查原始基线；保留失败证书，不直接跳过检查
Lean 下载失败	网络及工具链地址	按[5]修复安装；Python 实验可继续，形式状态保持未验收
Lean 编译报错	首条错误与固定版本	用 03 提示词逐个修复；不要换成额外公理
实验明显变慢	n 是否增大	回到 n≤10；一次仅增加 1 个状态，并记录时间
抽象下界很小	特征是否丢失位置	对照第 8 节；identity 恢复精确值，用 marked 作位置基线

排错时把完整文本输出复制到日志，不只保留截图或最后一行。一个错误必须先在最小输入上复现，再修改实现。修改检查器尤其需要重跑全部破坏测试，避免“修复”实际上取消了必要验证。

任何网络、环境或模型问题都不应改变数学目标。相反，若纸面证明发现错误，应保留原命题版本，并明确写出修改的假设或结论，重新运行相关实验。

算力、时间与现金安排

首轮计算仅使用 Python 标准库与 CPU；无需训练模型，也不要求显卡。建议使用一台日常笔记本，优先保持 n≤10。程序在准备环境的 14 项测试约 0.18 秒完成，这只是该环境单次记录，不能当作所有电脑的性能承诺。自己的环境与时间应由日志记录。

阶段	投入上限建议	达到什么再继续
首次复现	半天；不新增付费工具	示例、比较与测试全部运行
纸面审阅	1 个工作日	两锚点命题各边界清楚
形式化	2—3 个工作日	真实编译通过，语义对应明确
研究扩展	剩余窗口只选 1 个分支	先有完整定理或可证伪的精确猜测

现金预算按执行策略设置：第一轮新增支出可定为 0，使用已有电脑与已有工具。确需付费模型辅助时，可先设一个自行可接受的小额封顶，例如 100—300 元，并按每天的有效产物检查是否值得继续；这属于预算建议，未引用任何模型实际价格。

三条推进规则

连续两轮模型修改都没有通过同一条编译命令时，缩小文件，只保留产生错误的引理。一天内没有得到新的可复现实例或完整证明步骤时，停止增加实验规模，转回已具备纸面推导的命题。若 r 个锚点方向暂时没有一般公式，优先完成两锚点与旋转障碍的正式证明。

真正的投入记录包含数学判断时间、调试时间、构造证书时间和检查时间。只报模型 token 或 CPU 耗时，会漏掉选题与审阅的主要成本。

如何写成一份像样的研究成果

最终成果应围绕一个连贯问题组织：信息抽象如何限制最短同步证书。正文先给对象与问题，再给一般证书定理、信息受限障碍和具体闭式，随后展示实验，最后交代形式化与可复现性。使用 templates/RESULTS_REPORT.md 作为文章骨架。

建议的摘要结构

第一句描述研究对象；第二句说明采用了局部势函数与特征抽象；第三句只列已完成证明的结果；第四句给出确切实验范围；最后一句报告实际形式化状态。例如，只有 C_3 和 C_4 编译通过时，就准确写这两个固定实例，不将其扩写为任意 n 的内核证明。

你拥有的证据	可以使用的表述	需要补充的工作
复现 C_2—C_{10}	复现经典实例的精确阈值	不将经典结果认定为数学新发现
完整两锚点纸面证明	给出指定抽象的闭式推导	查重、审阅与 Lean 形式化
仅有 r 个锚点数表	观察到一组有限实例规律	完整量词、一般上下界证明
Python 证书通过	独立 Python 检查接受	Lean 形式环境及语义连接
真实 Lean 编译通过	列出具体形式定理及版本	按实际范围报告与接受外部复核

选题的新颖性可以来自新的数学刻画，也可以来自尚缺少的可复用形式化接口。两者需要不同的证据。首次性确认以前，使用“候选研究结果”“新形式化目标”等精确状态；完成查重与审阅后再调整。

四周扩展：从实例走向一般方法

赛事窗口结束后，项目可沿同一核心继续推进。第一周完成当前定义、证明与语义桥梁；第二周只研究固定 r=3；第三周研究缺少少量位置信息时的边界；第四周将证书接口推广到其他有限转移系统。每一步都有可独立验收的产物。

方向一：直接生成符号抽象图

当前代码先枚举 2^n 个集合再合并。对于固定 r 个锚点，特征由基数与 r 个成员位组成，候选类数至多$(n+1)2^r$。可以尝试直接根据计数与位置约束生成抽象边，避开完整幂集枚举。必须证明新边集与“存在真实代表”定义完全一致，尤其不能漏边导致下界虚高。

方向二：证书的信息复杂度

研究指定特征层级达到某个下界比例所需的 r。例如，在 C_n 上认证至少一半真实阈值，需要保留多少锚点？先固定锚点位置方案与证书类型，再提出量词完整的下界或上界。任何结论都应限定到所研究的表示与验证规则。

方向三：迁移证书原理

核心势函数定理适用于任何每步代价为 1 的有限转移系统。后续可研究受限规划、有限协议或带权最短路径，但每次只替换一个对象，保留检查器的信任结构。权重版本需要将“1”替换为明确的非负代价，并重新证明累加定理。

研究主线 始终追问“这份信息足够证明什么、缺少什么信息就一定证明不了什么”。这样每个新实验都有明确数学问题，而不会退化成不断扩大搜索规模。

来源、记录与最后的启动动作

[1] Open Problems Hack at MIT. Competition and Judging Handbook, September 2026. 本手册的赛程、M2 定位和提交流程依据。

https://rsihouse.ai/openmath/handbook.pdf

[2] Mikhail V. Volkov. List of Results on the Cěrný Conjecture and Reset Thresholds for Synchronizing Automata. arXiv:2508.15655, § 1.2. 用于同步定义、经典实例与背景。

https://arxiv.org/abs/2508.15655

[3] Marek Szykuła. Synchronizing Automata: Open Problems. arXiv:2608.24245, 2026. 用于领域研究问题的背景定位；未据摘要声称任何具体新突破。

https://arxiv.org/abs/2608.24245

[4] Python Documentation. Using Python on Windows. 使用官方发行与命令说明。

https://docs.python.org/3/using/windows.html

[5] Lean Language. Install. 本机工具安装入口。

https://lean-lang.org/install/

[6] Lean Community. Lean projects. 用于理解项目与版本环境。

https://leanprover-community.github.io/install/project.html

[7] Lean Language Reference. Tactic Reference. 用于查询有限判定与证明战术。

https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/

来源核对日期：2026 年 9 月 27 日。赛事信息具有时效性，以主办方最新通知为准。第 5—8 节给出的推导应与 docs/THEORY.md 逐项对应；其中所选抽象命题的文献新颖性仍需独立核验。实例数字与测试状态来自本包 reference_results，未引用或编造外部实验数据。

现在开始

打开项目根目录，执行下面的命令。先得到自己的第一份结果，再将今日目标写入日志。完成首次复现后，按照第 12 节锁定两锚点命题，推进真实 Lean 编译和完整语义对应。

```
python run.py demo
python run.py compare --n 8
python run.py landmarks --n 8
```

项目起点是一台普通电脑上的有限实例；研究目标是能够被独立重放、准确解释并继续扩展的数学证据。